

<MiniFarm>

2022182010 김정민

2020182045 황수지

1. 프로젝트 소개 (Project Introduction)

1.1. 프로젝트 개요

'MiniFarm'은 외딴 섬에 고립된 플레이어가 농작물을 재배하여 자금을 모으고, 배를 구매하여 섬을 탈출하는 것을 목표로 하는 3D 농장 시뮬레이션 및 상호작용 프로그램입니다. 사용자는 3차원 가상 공간에서 캐릭터를 조작하여 밭을 일구고 상점과 거래하며 목표를 달성해야 합니다.

1.2. 개발 목표 및 핵심 플레이

본 프로젝트는 사용자의 입력에 따라 프로그램의 상태가 실시간으로 변화하는 **상호작용** 구현에 중점을 두었습니다.

- **농장 운영:** 플레이어는 밭 갈기 → 씨앗 심기 → 물 주기 → 수확의 과정을 통해 작물(당근, 양배추)을 재배합니다.
- **경제 및 전략:** 작물마다 판매 가격이 다르므로, 플레이어는 목표 금액(배 구매 비용)을 효율적으로 달성하기 위해 전략적으로 농장을 운영해야 합니다.
- **탈출 엔딩:** 목표 금액을 달성하고 선착장의 보트와 상호작용하면 게임이 종료되는 명확한 목표 지향적 구조를 가집니다.

1.3. 주요 특징

- **모델링 및 렌더링:** OBJ 파일 로딩을 통해 플레이어, 작물, 건물 등 다양한 3D 객체를 렌더링하고 텍스처를 매핑하여 시각적 완성도를 높였습니다.
- **조명 및 환경:** phong 조명(Phong Lighting) 모델을 기반으로, 게임 내 시간 흐름(TimeSystem)에 따라 낮과 밤이 변화하는 조명 효과를 구현했습니다.
- **특수 효과:** 물뿌리개 사용 시 Geometry Shader를 활용한 물 파티클(Particle) 효과를 구현하여 시각적 피드백을 강화했습니다.

2) 시스템 상호작용 구현

- 성장 시스템:** 모든 작물은 4단계의 성장 과정을 가지며, 시간 경과와 물 주기 여부에 따라 상태가 실시간으로 변화합니다.
- UI 시스템:** 3D 화면 위에 2D 인터페이스(인벤토리, 상점 대화창, 돈 표시)를 오버레이하여 게임 정보를 직관적으로 제공합니다.
- 데이터 주도 설계:** 맵 배치와 게임 밸런스 데이터를 외부 파일(JSON, CSV)로 관리하여 프로그램의 유연성을 확보했습니다.

2. 구조 설명: 프로젝트 클래스 구조

2.1. 프로그램 아키텍처 개요

본 프로젝트 MiniFarm은 유지보수와 협업의 용이성을 위해 Core(엔진/코어), Game(게임 로직), Render(렌더링), Data(데이터) 모듈로 명확히 분리된 구조를 채택하였습니다.

핵심 설계 특징

- 모듈화:** 기능별 폴더 분리를 통해 코드 의존성을 최소화했습니다.
- 객체지향 (OOP):** GameObject를 상속받는 구조로 확장성을 확보했습니다.
- 데이터 주도 (Data-Driven):** 맵 배치와 게임 밸런스를 외부 파일(JSON, CSV)로 관리합니다.

2.2. 폴더별 상세 클래스 명세

1. Core

게임의 실행 흐름과 입력을 관리하는 핵심 모듈입니다.

파일/클래스	역할 및 기능 설명
Main.cpp	프로그램 진입점(Entry Point). 윈도우 생성 및 메인 루프 실행.
SceneManager	Title ↔ Ingame ↔ Exit 씬 전환 및 생명주기(Life-cycle) 관리.
InputManager	키보드 및 마우스 입력 상태를 감지하고 처리.

2. Game

실제 게임 플레이와 상호작용 로직이 포함된 모듈입니다.

파일/클래스	역할 및 기능 설명
IngameScene	게임의 메인 로직 루프가 실행되는 씬. 모든 객체의 업데이트를 총괄.
GameObject	모든 게임 객체의 최상위 부모 클래스. 위치(Pos), 회전(Rot) 데이터 보유.
Player	사용자 입력을 받아 이동하며, 도구(호미, 물뿌리개 등) 사용 상태 관리.
Inventory	플레이어의 아이템(수확물) 및 재화(Money) 데이터 관리.
StaticProp	상호작용이 없는 배경 오브젝트 (집, 울타리, 나무 등) 처리.

상호작용 객체 구조 (Inheritance)

상호작용 가능한 객체들은 `IInteractable` 인터페이스와 `InteractableObject` 클래스를 통해 구현되었습니다.

- **IInteractable (Interface):** 상호작용 메서드를 정의한 인터페이스.
- **InteractableObject:** `GameObject`를 상속받고 `IInteractable`을 구현한 중간 부모 클래스.
 - **Crop:** 4단계 성장 로직, 물 주기/수확하기 기능 구현.
 - **Shop:** 작물별 가격 데이터를 기반으로 구매/판매 기능 구현.
 - **Boat:** 탈출 목표(금액) 달성 여부 체크 및 게임 엔딩 처리.

3. Render

OpenGL API를 캡슐화하여 화면 출력을 담당하는 모듈입니다.

파일/클래스	역할 및 기능 설명
Camera	3인칭 시점의 View/Projection 행렬 계산 및 카메라 이동 제어.
Model	.obj 파일을 파싱하여 정점, 법선, UV 데이터를 로드.
TextureLoader	이미지 파일(.png)을 로드하여 텍스처 객체 생성.
Shader	GLSL 셰이더 컴파일 및 유니폼(Uniform) 변수 핸들링.
UIRenderer	2D UI 스프라이트(인벤토리 바, 아이콘 등) 렌더링.
TextRenderer	게임 내 텍스트(가격 표시, 안내 문구) 렌더링.

WaterParticleSystem	Geometry Shader를 활용한 물 입자 파티클 효과 생성.
---------------------	--------------------------------------

4. Systems / Utils / Data

- **Systems (TimeSystem):** 게임 내 시간 흐름(낮/밤) 관리 및 조명 값 업데이트.
- **Utils (MathHelper):** 벡터 연산, 난수 생성 등 보조 함수 모음.
- **PCH (pch.h):** Precompiled Header를 통한 빌드 속도 최적화.
- **Data (Resources):**
 - crops.csv: 작물 성장 시간, 판매 가격 밸런스 데이터.
 - static_props_pos.json: 맵 정적 오브젝트 좌표 데이터.

2.3. 주요 데이터 흐름 (Data Flow)

1. **초기화 (Initialization):** SceneManager가 IngameScene을 로드할 때 Model과 리소스를 메모리에 올리고, json 파일을 읽어 맵을 배치합니다.
2. **업데이트 (Update Loop):** 매 프레임 InputManager의 입력을 받아 Player를 이동시키고, Crop 등의 게임 로직을 업데이트합니다.
3. **렌더링 (Render Loop):** Camera 시점을 기준으로 3D 월드를 렌더링한 후, UIRenderer가 UI를 최상단에 덧그려서 최종 화면을 완성합니다.

3. 프로젝트 진행

3.1. 팀원 간 작업 내용

주차	김정민 (플레이어/그래픽/렌더링)	황수지 (시스템/상호작용 로직)
1주차	기본 큐브형태 객체 배치, 3인칭 카메라 + 플레이어 이동 구현.	충돌 처리 구현, 인벤토리/도구 교체 및 작물 성장 (상태머신) 기본 구조 구현.
2주차	OBJ 모델 로딩 (집, 작물 4단계 등), 텍스처 매핑 시도, 태양 조명 구현.	오브젝트 위치 JSON 추출, 충돌 처리 개선, 마우스 클릭 기반 상호작용 시스템 초안 구현.
3주차	좌표계/좌표 버그 수정, 땅/바다 분리, 플레이어/도구 애니메이션 (5개 파트) 구현.	작물 상호작용 로직 완성, 위치 기반 상점 거래 시스템 마무리, UI 표시 및 보트 상호작용 구현.
4주차	밤낮 조명 시스템 구현 및 시간 시스템 연동, 물뿌림 파티클 연출 구현.	탈출 씬 구현, 작물 성장에 시간 시스템 적용 및 Time Bar UI 구현.

4. 필요한 명령어 소개

분류	명령어	기능
이동	W, A, S, D	플레이어의 전후좌우 이동
시점	마우스 이동	3인칭 카메라 시점 회전
도구 전환	숫자 키 (1 ~ 5)	1: 호미 (땅 갈기), 2: 물뿌리개 (물 주기) , 3: 낫 (수확), 4: 당근 씨앗, 5: 양배추 씨앗
상호작용	E	밭 상호작용, 상점 입장, 보트 상호작용 (탈출)
시연용	C	돈 10,000 획득 (탈출 목표 금액 즉시 달성용 치트키)
종료	ESC	프로그램 즉시 종료
상점	S, B, X	작물 판매, 씨앗 구매, 닫기
탈출	Y, N	탈출 조건 달성 시 선택(예/아니오)

5. 프로젝트 개발 소감 및 후기

김정민 (플레이어/그래픽/렌더링)

이번 프로젝트의 가장 큰 수확은 '좌표계'와 씨름한 경험이었습니다. 유니티 같은 상용 엔진에서 가져온 OBJ 모델들이 자체 엔진에서는 뒤집혀 보이거나 전혀 엉뚱한 위치에 렌더링되는 문제가 반복적으로 발생하여, 이를 하나씩 맞춰 가는 과정이 쉽지 않았습니다. 그러나 이 과정을 통해 3D 공간을 컴퓨터가 어떤 기준과 순서로 계산하는지 보다 명확하게 이해할 수 있었습니다. 특히 해가 지고 밤이 되는 조명 효과나, 물뿌리개 사용 시 물방울이 튀는 파티클 효과가 의도한 대로 화면에 구현되었을 때 큰 성취감을 느낄 수 있었습니다. 작성한 코드가 단순한 텍스트를 넘어 실제 '움직이는 세계'로 표현된다는 점이 그래픽스 프로그래밍의 가장 큰 매력이라고 생각합니다.

황수지 (시스템/상호작용 로직)

화면에 보이는 그래픽보다는 그 뒤에서 동작하는 내부 로직을 탄탄하게 설계하고 구현하는 데에 집중하였습니다. 처음에는 기능이 늘어날수록 코드 복잡도가 높아지는 것이 부담이었으나, 상속 구조(부모-자식 클래스)를 먼저 정리해 두니 작물, 상점, 보트 등의 기능을 확장할 때 훨씬 수월해졌습니다. 특히 씨앗이 자라 수확물이 되기까지의 4단계 성장 과정이 버그 없이 자연스럽게 동작할 때 높은 만족감을 얻을 수 있었습니다. 데이터 관리 방식 개선도 중요한 성과였습니다. 게임 밸런스나 나무 위치를 변경할 때마다 코드를 직접 수정하는 비효율을 줄이기 위해, 관련 정보를 엑셀(CSV)과 JSON 파일로 분리하여 관리하도록 구조를 변경하였습니다. 이로 인해 수정 작업 시간이 크게 단축되었습니다. 또한 팀원과의 역할 분담을 명확히 하고 각자 담당 영역을 존중하며 협업함으로써, 큰 충돌 없이 프로젝트를 안정적으로 마무리할 수 있었습니다.